

Cross-Phase Dependency Graph — the HelixVPN delivery DAG and critical paths

Cross-Phase Dependency Graph — the HelixVPN delivery DAG and critical paths

Revision: 1 Last modified: 2026-06-26T12:00:00Z

Volume 7 (Phase Execution), document 2 of 5. This spec consolidates the dependency edges scattered across the four phase WBS documents into one **cross-phase directed acyclic graph (DAG)**: which Phase-0 spikes gate which Phase-1 epics, the critical path to the MVP tag, the cross-component dependencies that bind data-plane [?] control-plane [?] client, and the parallel fan-outs that compress wall-clock. It is **spec-only** — every edge is cited to a `depends_on/deps` field in `06-phase0-spike-wbs.md` (HVPN-P0-NNN), `07-phase1-mvp-wbs.md` (HVPN-P1-NNN), `08-phase2-parity-wbs.md` (HVPN-P2-NNN), or `09-phase3-reach-wbs.md` (HVPN-P3-NNN); nothing is invented. The DAG is the structure the `workable-items` loader validates for acyclicity (§11.4.93, see `workable-items-model.md` §8). All durations are sizing-only TARGETs — never date commitments (§11.4.6, the phase docs' no-commitment caveat).

Table of contents

- 0. Reading the DAG
 - 1. Programme-level phase DAG
 - 2. Phase 0 → Phase 1 gating (spikes that unblock epics)
 - 3. The Phase-1 MVP critical path
 - 4. Cross-component dependencies (data [?] control [?] client)
 - 5. Phase-2 critical path + parallel high-risk tracks
 - 6. Phase-3 critical path + device-gated parking
 - 7. Master dependency table (item → depends-on → blocks)
 - 8. Critical-path call-out + parallelism rules
 - Sources verified
-

0. Reading the DAG

An edge $A \rightarrow B$ means **B depends on A** (A must be complete before B starts), sourced from B's depends_on/deps field in its WBS doc. Cross-phase edges cite the full id (08-... §0). The graph is acyclic by construction; `workable-items validate` (`workable-items-model.md` §8) fails the build on any cycle.

Risk-ordering (§11.4.132) governs *sequencing within* the DAG: the irreversible correctness floor (RLS, no-log lint, kill-switch, NAT-traversal honesty, PQ downgrade-safety) leads; convenience UI trails. Red nodes are the critical-path / make-or-break carries each phase doc flags.

1. Programme-level phase DAG

```
graph LR
  P0["Phase 0 - Spike<br/>G1-G6 gates<br/>HVPN-P0-001..080"]
  P1["Phase 1 - MVP<br/>8 AC + 4 SL0<br/>HVPN-P1-010..151"]
  P2["Phase 2 - Parity+Reach<br/>8 P2-AC + 5 SL0<br/>HVPN-P2-190..301"]
  P3["Phase 3 - Extended Reach<br/>G20-G26 gates<br/>HVPN-P3-200..272"]
  P0 -->|"G1/G2/G4/G5/G6 settled<br/>before P1 starts"| P1
  P1 -->|"8 DoD + 4 SL0 green<br/>= seams exist (P2-E19)"| P2
  P2 -->|"§08 DoD green<br/>= P3 entry condition"| P3
  classDef crit fill:#fde,stroke:#c33,stroke-width:2px;
  class P0,P1 crit;
```

Each phase boundary is a hard gate (07-... §5, 08-... §5, 09-... §2.0): the make-or-break Phase-0 carries (G3 iOS memory, G4 edge language) are *resolved before* Phase 1 starts (07-... §4); Phase 2 assumes the Phase-1 seams exist (08-... §5 entry gate HVPN-P2-190/191/192); Phase 3 starts only when Phase 2 has graduated (09-... §2.0).

2. Phase 0 → Phase 1 gating (spikes that unblock epics)

The six Phase-0 gates map to Phase-1 entry items HVPN-P1-001..006 (07-... §5), which in turn gate the Phase-1 epics. The dependency-bearing edges (from the Phase-1 deps fields):

```
graph TD
  G1["G1 plain-UDP data-path<br/>HVPN-P0-025 → HVPN-P1-001"]
  G2["G2 MASQUE-through-DPI<br/>HVPN-P0-035 → HVPN-P1-002"]
  G3["G3 iOS NE memory (make-or-break)<br/>HVPN-P0-061..070 → HVPN-P1-003"]
  G4["G4 edge language<br/>HVPN-P0-045 → HVPN-P1-004"]
  G5["G5 FFI boundary<br/>HVPN-P0-052 → HVPN-P1-005"]
  G6["G6 push-reconcile<br/>HVPN-P0-071..076 → HVPN-P1-006"]
```

```

G5 --> E01["E01 Foundation<br/>HVPN-P1-010 deps HVPN-P1-005"]
G2 --> E09T["HVPN-P1-090 deps HVPN-P1-002<br/>(Transport
trait+MASQUE)"]
G4 --> E10["HVPN-P1-100 deps HVPN-P1-004<br/>(edge language
fixed)"]
G3 --> E12i["HVPN-P1-122 deps HVPN-P1-003<br/>(iOS NE within
budget)"]
G1 --> E09T
E01 --> E02["E02 Store+RLS"]
classDef mk fill:#fdd,stroke:#c33,stroke-width:2px;
class G3 mk;

```

Phase-0 gate	Decides	Phase-1 entry item	Phase-1 epic it unblocks
G1 plain-UDP slice (HVPN-P0-025)	core viability	HVPN-P1-001	E09 (HVPN-P1-090 Transport)
G2 MASQUE through DPI (HVPN-P0-035)	obfuscation D1	HVPN-P1-002	E09 HVPN-P1-090, E10 HVPN-P1-100
G3 iOS NE memory (HVPN-P0-061..070)	Rust-on-every-platform D2	HVPN-P1-003	E12 HVPN-P1-122 (iOS shim)
G4 Go-vs-Rust edge (HVPN-P0-045)	edge language D5	HVPN-P1-004	E10 HVPN-P1-100 (fixes edge lang)
G5 frb FFI (HVPN-P0-052)	FFI boundary	HVPN-P1-005	E01 HVPN-P1-010 (monorepo)
G6 push-reconcile (HVPN-P0-071..076)	push model seed	HVPN-P1-006	E07 WatchNetworkMap seed

G3 fail-path (§11.4.66). If the iOS memory gate fails, HVPN-P1-122 (iOS shim) re-plans against the §6.4 fallback ladder (07-... §5 note) — a decision surfaced, not silently resolved.

3. The Phase-1 MVP critical path

The longest dependency chain to AC certification (07-... §22), with the red critical-path epics E02/E03/E07/E09/E14 (07-... §4):

```

graph LR
  A["E01 010→011<br/>repo+proto"] --> B["E02 020→021→022<br/>store+FORCE-RLS"]
  B --> C["E03 032<br/>PKI device-cert"]
  C --> D["E07 070→071→072→073<br/>coordinator+WatchNetworkMap"]
  D --> E["E09 092→093/094<br/>orchestrator+kill-switch"]
  E --> F["E10 100→101→102<br/>edge+verdict-map"]
  F --> G["E14 140→142→144→145<br/>QA+DoD cert"]
  G --> H["E15 151<br/>release tag"]
  classDef crit fill:#fde,stroke:#c33,stroke-width:2px;
  class B,C,D,E,G crit;

```

Critical path: E01(010→011) → E02(020→021→022) → E03(032) → E07(070→071→072→073) → E09(092→093/094) → E10(100→101→102) → E14(140→142→144→145) → E15(151) (07–... §22). The two parallel fan-outs that keep wall-clock down: **E04/E05/E06** off E02 (disjoint scope → concurrent PWUs §11.4.58), and **E11 UI** off E08/E09 in parallel with E10/E12.

4. Cross-component dependencies (data control client)

The architecture's three planes (data / control / client) are bound by specific deps edges. The MVP is hub-and-spoke: the coordinator computes per-node maps, the edge enforces them, the client core reconciles them.

```
graph TD
  subgraph CONTROL["Control plane (Go)"]
    STORE["E02 store+RLS<br>HVPN-P1-022"]
    POLICY["E06 policy compiler<br>HVPN-P1-061 (2 artifacts)"]
    COORD["E07 coordinator<br>HVPN-P1-071 buildMap<br>HVPN-P1-072 WatchNetworkMap"]
    API["E08 API+authz<br>HVPN-P1-080 mTLS resolver"]
  end
  subgraph CLIENT["Client core (Rust)"]
    TRANS["E09 Transport+MASQUE<br>HVPN-P1-090"]
    ORCH["E09 orchestrator+reconciler<br>HVPN-P1-092"]
    KILL["E09 kill-switch<br>HVPN-P1-094"]
  end
  subgraph DATA["Data plane (Rust edge)"]
    EDGE["E10 MASQUE termination<br>HVPN-P1-100"]
    PEER["E10 kernel-WG peer prog<br>HVPN-P1-101"]
    VERD["E10 verdict-map enforce<br>HVPN-P1-102"]
  end
  STORE --> POLICY --> COORD
  COORD --> API
  COORD --> ORCH
  POLICY --> VERD
  TRANS --> ORCH --> KILL
  TRANS --> EDGE --> PEER --> VERD
  COORD --> PEER
  COORD --> VERD
  classDef crit fill:#fde,stroke:#c33;
  class COORD,TRANS crit;
```

The load-bearing cross-plane edges (from the deps fields):

Edge	Cited dep	Why it binds the planes
policy compiler → coordinator	HVPN-P1-070 deps HVPN-P1-061	buildMap consumes the compiled {visible,allowedIPs,verdict-map} (07–... §11/§12)
	HVPN-P1-092 deps HVPN-P1-072	the client converges to streamed MapUpdate deltas (07–... §14)

Edge	Cited dep	Why it binds the planes
coordinator → client reconciler		
Transport trait → edge	HVPN-P1-100 deps HVPN-P1-090,004	edge reuses helix-transport byte-for-byte; G4 fixes language (07-... §15)
compiler verdict-map → edge	HVPN-P1-102 deps HVPN-P1-061,101,033	edge applies the port-level verdict map WG AllowedIPs lacks (07-... §15)
PKI cert → API/ coordinator authz	HVPN-P1-080 deps HVPN-P1-032; HVPN-P1-072 auth via mTLS	the device cert authenticates the stream (07-... §12/§13)
IPAM → enroll	HVPN-P1-031 deps HVPN-P1-040	enroll allocates the overlay IP before issuing the map (07-... §8)

5. Phase-2 critical path + parallel high-risk tracks

Phase 2 is additive off the shipped MVP (08-... §4). The networking/reach spine is the critical path; PQ and DAITA are parallel measure-don't-assert tracks.

graph TD

```

P1["Phase 1 shipped"] --> E27["E27 HA+multi-region<br/>270→271→272→273→274"]
E27 --> E22["E22 P2P+NAT traversal<br/>220→221→222→223→224→225"]
E22 --> E23["E23 multi-hop<br/>230→231→232"]
E23 --> E29["E29 QA+DoD cert<br/>290→295→297"]
E29 --> E30["E30 release tag 301"]
P1 -.parallel.-> E24["E24 post-quantum<br/>240→241→242→243"]
P1 -.parallel.-> E21["E21 DAITA<br/>210→211→213"]
P1 -.parallel.-> E20["E20 transport set<br/>200..206"]
P1 -.parallel.-> E25["E25 desktop apps"]
P1 -.parallel.-> E26g["E26 GitOps"]
E24 --> E29
E21 --> E29
E20 --> E29
classDef crit fill:#fde,stroke:#c33,stroke-width:2px;
class E27,E22,E23,E29 crit;
```

Critical path:

E27(270→271→272→273→274) → E22(220→221→222→223→224→225) → E23(230→231→232) → E29(290→295→297) → E30(301) (08-... §19). E27 leads because the STUN endpoints, relay fleet, and NATS mesh that E22/E26/E28 consume are *its* outputs (08-... §4): HVPN-P2-220 (STUN discovery) deps HVPN-P2-271; HVPN-P2-223 (relay) deps HVPN-P2-273. The two high-risk parallel tracks that converge at E29 but do **not** sit on the networking critical path: **E24 (PQ)** and **E21 (DAITA)** — each a disjoint-scope PWU (§11.4.58). Per §11.4.132 the correctness-and-downgrade-safety floor leads: NAT-traversal honesty (HVPN-P2-222 no-false-direct), PQ never-weaker-than-classical (HVPN-P2-242), DAITA measured efficacy (HVPN-P2-213).

6. Phase-3 critical path + device-gated parking

Phase 3's two native shims (E21 HarmonyOS, E22 Aurora) are the make-or-break carries; they run as disjoint-scope parallel PWUs the moment E20's fork runners exist, and *park* on PENDING_DEVICE while host-only work proceeds (09–... §4).

graph TD

```
P2["Phase 2 graduated"] --> E20["E20 reach CI fabric<br/>200/201→202→203 · G20"]
E20 --> E21["E21 HarmonyOS<br/>210→211→214 · G21 ▲"]
E20 --> E22["E22 Aurora<br/>220→221→223 · G22 ▲"]
E20 --> E23["E23 WASM proxy<br/>230→231→233 · G23"]
E20 --> E24["E24 billing<br/>240→241..244 · G24"]
E21 --> E25["E25 3rd-party audit<br/>250→252→253→254 · G25"]
E22 --> E25
E23 --> E25
E24 --> E25
E25 --> E26["E26 reproducible builds<br/>260→263→264 · G26"]
E26 --> E27["E27 l10n+release<br/>271→272"]
E21 --> E27
E22 --> E27
classDef risk fill:#fdd,stroke:#c33,stroke-width:2px;
class E21,E22 risk;
```

Critical path (risk-ordered §11.4.132):

E20(200/201→203) → {E21(210→211→214) // E22(220→221→223)} →
E25(250→252→253→254) → E26(260→263→264) → E27(271→272) (09–... §14).
E25 (audit) is a **convergence gate** — it cannot start until the surfaces it audits (E21–E24) are feature-complete (HVPN–P3–250 deps 214, 223, 233, 244); E26 depends on E25 because the audited source tree is what must reproduce. While E21/E22 await device access (PENDING_DEVICE), E23/E24/E26-tooling progress as ≥3 background streams (§11.4.103, 09–... §4).

7. Master dependency table (item → depends-on → blocks)

The load-bearing edges per phase (selected critical/cross-component edges; the full set is every deps/depends_on field in the four WBS docs, which the loader ingests).
blocks = the inverse edge (what waits on this item).

Item	Depends on	Blocks (selected)	Source
HVPN–P0–004 Transport trait	HVPN–P0–002	HVPN–P0–008,028,049	06–... §3
HVPN–P0–008 plain-UDP	HVPN–P0–006	HVPN–P0–018 (orchestrator)	06–... §3
HVPN–P0–025 G1 reach	HVPN–P0–020,023	HVPN–P1–001 (G1 entry)	06–... §5

Item	Depends on	Blocks (selected)	Source
HVPN-P0-035 G2 MASQUE	HVPN-P0-031, 024	HVPN-P1-002, HVPN-P0-039 (Rust edge)	06-... §6
HVPN-P0-045 G4 bench	HVPN-P0-040, 043	HVPN-P1-004 (edge lang)	06-... §7
HVPN-P1-010 monorepo	HVPN-P1-005 (G5)	HVPN-P1-011, 013, 110, 150	07-... §6
HVPN-P1-022 FORCE-RLS	HVPN-P1-021	HVPN-P1-030, 032, 040, 050, 060	07-... §7
HVPN-P1-061 policy compiler	HVPN-P1-060	HVPN-P1-062, 070, 102	07-... §11
HVPN-P1-072 WatchNetworkMap	HVPN-P1-071, 080	HVPN-P1-073, 092, 101	07-... §12
HVPN-P1-090 Transport (prod)	HVPN-P1-002	HVPN-P1-091, 093, 100	07-... §14
HVPN-P1-100 edge MASQUE term	HVPN-P1-090, 004	HVPN-P1-101, 102, 261	07-... §15
HVPN-P1-145 DoD cert	140, 141, 142, 143, 144	HVPN-P1-151 (tag)	07-... §19
HVPN-P2-190 Transport seam	HVPN-P1-090	HVPN-P2-200, 201, 202	08-... §5
HVPN-P2-220 STUN discovery	HVPN-P2-191, 271	HVPN-P2-221, 222, 296	08-... §8
HVPN-P2-222 hole-punch	HVPN-P2-220, 221	HVPN-P2-223, 224, 290	08-... §8
HVPN-P2-240 ML-KEM PSK	HVPN-P2-192	HVPN-P2-241, 244	08-... §10
HVPN-P2-272 NATS mesh	HVPN-P2-270	HVPN-P2-273, 281, 295	08-... §13
HVPN-P2-297 Phase-2 DoD	290..296	HVPN-P2-301 (tag)	08-... §15
HVPN-P3-200/201 fork runners	—	HVPN-P3-210, 220, 230, 260	09-... §5
HVPN-P3-210 OHOS cross-compile	HVPN-P3-200	HVPN-P3-211	09-... §6
HVPN-P3-250 audit scope	214, 223, 233, 244	HVPN-P3-252, 253	09-... §10
HVPN-P3-260 determinize builds	HVPN-P3-202	HVPN-P3-261, 262, 263	09-... §11

8. Critical-path call-out + parallelism rules

The single longest chain to a published release (programme critical path, concatenating the per-phase critical paths at the gate boundaries):

Phase 0: HVPN-P0-002 → 004 → 008 → 018 → 025(G1) → 031 → 035(G2) → 045(G4)
 Phase 1: → P1-010 → 022 → 032 → 070 → 071 → 072 → 092 → 094 → 100 → 102 → 145 → 151(MVP tag)
 Phase 2: → P2-270 → 271 → 272 → 220 → 222 → 223 → 224 → 230 → 232 → 295 → 297 → 301(parity tag)
 Phase 3: → P3-200 → 203(G20) → 210 → 211 → 214(G21) → 250 → 252 → 253 → 254(G25) → 260 → 263 → 264(G26) → 272(reach tag)

Parallelism rules that compress this wall-clock (each cited to a constitution anchor + the WBS note that invokes it):

Rule	Where it applies	Anchor
Disjoint-scope PWUs run fully parallel	P1 E04/E05/E06 off E02; P2 E20/E21/E24/E25; P3 E23/E24 off E20	§11.4.58
≥3 background streams + auto-backfill	every phase; P3 while shims park on PENDING_DEVICE	§11.4.103
Single-resource-owner for shared hardware	P0 S4 vs S7 must NOT share the bench host at once	§11.4.119 (06-... §2 note)
Risk-ordered sequencing (highest-risk first)	RLS/kill-switch/NAT-honesty/PQ-downgrade-safety lead	§11.4.132
Audio-style top-priority maps to correctness floor	every phase's iteration ordering	§11.4.72→§11.4.132

Honest-uncertainty markers (§11.4.6). Phase-3 device-gated nodes (HVPN-P3-211/221 and downstream) carry the widest error bars and are UNCONFIRMED:/PENDING_DEVICE: until hardware exists (09-... §14 risk register). A gate that cannot be cleared on the real device **is the finding** — escalate per §11.4.66/.101/.112, never overrun silently. No edge in this DAG implies a calendar date; every duration is a sizing-only TARGET.

Sources verified

- 06-phase0-spike-wbs.md §2 milestone DAG + §2 parallelism note, §3-§11 per-milestone depends_on fields, gates table — read 2026-06-26.
- 07-phase1-mvp-wbs.md §4 epic DAG, §5 entry gates, §21 traceability, §22 critical path + effort caveat — read 2026-06-26.
- 08-phase2-parity-wbs.md §4 epic DAG, §5 entry seams, §18 traceability, §19 critical path — read 2026-06-26.
- 09-phase3-reach-wbs.md §2.1 gate DAG, §4 epic DAG, §13 traceability, §14 critical path + risk register — read 2026-06-26.
- Sibling workable-items-model.md (§8 loader acyclicity validation) — authored this volume.
- Constitution anchors §11.4.58/.66/.72/.101/.103/.112/.119/.132 — read 2026-06-26.

Honest boundary (§11.4.6): every edge is transcribed from a cited WBS deps field; no dependency is inferred or invented. Durations are sizing TARGETs, never commitments; device-gated Phase-3 chains remain PENDING_DEVICE.